

HelixPlay

HelixPlay

Convierte cualquier máquina GPU en tu propio dispositivo de juegos en la nube.

Resumen

HelixPlay es una plataforma de juegos en la nube autohospedable que transforma cualquier máquina equipada con GPU en un servidor de streaming remoto, ofreciendo una experiencia de juego al nivel de consola en clientes de escritorio, móviles, televisores y navegadores. Está construida como un monorrepositorio centrado en Go, compuesto por 46 submódulos, con un cliente de triple pila y es personalizable para socios bajo marca blanca.

Descripción breve

HelixPlay es una plataforma de juegos en la nube autohospedable, abierta y personalizable bajo marca blanca. Convierte cualquier máquina equipada con GPU en un servidor de streaming remoto y ofrece una experiencia de juego al nivel de consola en clientes de escritorio, móviles, televisores y navegadores a través de WebRTC/QUIC, con un núcleo basado en Go y una pila de cliente Wails/Flutter/Angular.

Descripción detallada

HelixPlay es una plataforma de juegos en la nube construida como un monorrepositorio centrado en Go, compuesto por 46 submódulos de Git. Toma cualquier PC para juegos que ya poseas y la convierte en un servidor de streaming, ofreciendo una experiencia al nivel de consola en clientes de escritorio, móviles, televisores y navegadores: autohospedable, abierta y

personalizable bajo marca blanca para socios. La propuesta es clara: tu hardware, tu servicio, tu marca, sin nubes de terceros de por medio.

Su decisión de diseño más distintiva es la convergencia de clientes en una triple pila, una apuesta arquitectónica que rinde frutos en todos los demás aspectos. Una aplicación de escritorio Wails, una app móvil/TV Flutter y un cliente web Angular se ejecutan sobre un *único* núcleo Go, compilado a WASM para el navegador, de modo que el comportamiento se escribe una sola vez y se comparte en todas las plataformas, en lugar de bifurcarse en tres versiones. Bajo esta capa se encuentra la ruta de medios en tiempo real: Captura → Codificación → Paquetización → Transmisión → Decodificación → Renderizado, conectada a capturas nativas de la plataforma (DXGI / ScreenCaptureKit / PipeWire) y codificadores por hardware (NVENC / QSV / AMF / VideoToolbox), de manera que el GPU asume la carga pesada. El transporte se realiza a través de WebRTC (Pion v4), QUIC (quic-go) y datagramas personalizados UDP, seleccionados por su baja latencia en lugar de por comodidad. El núcleo del backend gestiona sesiones, inquilinos, catálogo y autenticación; un agente en el host se encarga de la captura, codificación y transporte en el extremo; y mDNS/rendezvous unifica el descubrimiento para que los clientes encuentren su host sin configuración manual.

HelixPlay está diseñado desde cero para SaaS de marca blanca: tematización por inquilino, filtrado de catálogo, OAuth2 y facturación, de modo que un socio pueda implementar un servicio completamente personalizado en lugar de una simple adaptación superficial. Además, es nativo de contenedores en todos los detalles: cada servicio, base de datos, compilación, prueba y análisis se ejecuta dentro de contenedores, lo que hace que toda la plataforma sea reproducible tanto en despliegue como en verificación. Al igual que el resto de la familia Helix, se rige por una constitución anti-engaño en la que una prueba exitosa garantiza un comportamiento real y utilizable por el usuario final, no un simple *mock* que pasa.

Por qué lo creamos

El *cloud gaming* comercial es cerrado, centralizado y alquilado. HelixPlay se creó para que cualquiera con una máquina GPU pueda ejecutar su propio servidor de streaming —abierto, autohospedado y personalizable bajo marca blanca— en lugar de depender de un servicio de terceros.

Por qué es un cambio de juego

Fusiona tres elementos que los servicios comerciales mantienen separados: el autoalojamiento en hardware bajo tu control, un único núcleo Go que impulsa tres pilas de clientes para que las funciones lleguen a todos los entornos al mismo tiempo, y la multitenencia de marca blanca. El resultado es que un socio puede lanzar un servicio de *cloud gaming* completamente personalizado en sus *propias* GPUs —controlando la experiencia, los usuarios y la economía—, en lugar de revender capacidad en la nube de otro y vivir dentro de sus limitaciones.

Qué lo hace innovador

- **Convergencia de triple pila de clientes:** Wails, Flutter y Angular funcionando sobre un único núcleo Go (WASM en el navegador), de modo que escritorio, móvil, TV y web comparten una sola implementación en lugar de tres versiones divergentes.
- **SaaS autoalojable y de marca blanca:** tematización por inquilino, filtrado de catálogo, OAuth2 y facturación integrados, lo que permite que la plataforma se distribuya como un producto personalizable, no como una demo.
- **Transporte de baja latencia moderno:** WebRTC (Pion), QUIC y un UDP personalizado, combinados con selección de codificadores por hardware según la plataforma (NVENC / QSV / AMF / VideoToolbox), optimizados para la capacidad de respuesta por encima de la comodidad.
- **Arquitectura desacoplada de 46 submódulos:** componentes claramente separados con todo nativo de contenedores: cada servicio, base de datos, compilación, prueba y análisis se ejecuta en un contenedor.

Principales desafíos técnicos y cómo los resolvimos

- **Transmisión de baja latencia en hardware heterogéneo.** Cada sistema operativo y GPU expone la captura y la codificación de manera distinta, y la latencia no perdona. Se resolvió con una ruta de captura/codificación consciente de la plataforma —DXGI / ScreenCaptureKit / PipeWire alimentando NVENC / QSV / AMF / VideoToolbox—, transportada mediante WebRTC / QUIC / UDP para que cada máquina utilice su ruta nativa más rápida hacia los píxeles.
- **Un solo producto para escritorio, móvil, TV y web.** Se resolvió con clientes de triple pila (Wails, Flutter, Angular) que comparten un único núcleo Go compilado a WASM para el navegador, de modo que una corrección o función

implementada una vez aparece en las cuatro plataformas en lugar de requerir cuatro adaptaciones.

- **Operación multitenencia con marca blanca.** Se resolvió integrando tematización por inquilino, filtrado de catálogo, OAuth2 y facturación directamente en el núcleo del *backend*, de manera que el aislamiento entre inquilinos y la personalización de marca son primitivas de la plataforma, no bifurcaciones por cliente.

Pila tecnológica

- **Go (raíz 1.26.2 / submódulos 1.25+):** el núcleo compartido del *backend* y el agente anfitrión; un único lenguaje que se compila tanto a binarios nativos como a WASM, lo que hace posible el diseño de núcleo único y múltiples clientes.
- **Wails v2:** el cliente de escritorio, que vincula el núcleo Go a una *webview* integrada para que la aplicación de escritorio reutilice la lógica del núcleo en lugar de volver a implementarla.
- **Flutter 3.29+:** el cliente para móvil/TV, que invoca el núcleo Go mediante FFI para ofrecer una interfaz nativa en teléfonos y televisores sin necesidad de un segundo *backend*.
- **Angular 17+:** el cliente web, que ejecuta el mismo núcleo Go compilado a WASM, de modo que el navegador es una superficie de primera clase, no una versión reducida.
- **WebRTC / Pion v4, QUIC / quic-go, UDP personalizado:** tres protocolos de transporte en tiempo real para que la plataforma elija la ruta de menor latencia disponible según la red y el cliente.
- **Codificadores por hardware (NVENC / QSV / AMF / VideoToolbox) y captura por plataforma (DXGI / ScreenCaptureKit / PipeWire):** la ruta de captura y codificación acelerada por GPU, seleccionada por plataforma para que la codificación nunca se convierta en un cuello de botella en la CPU.
- **Contenedores (Docker/Podman):** cada servicio, base de datos, compilación, prueba y análisis se ejecuta en contenedores, lo que hace que todo el sistema sea reproducible tanto en despliegue como en verificación.
- **mDNS / rendezvous:** descubrimiento de anfitriones sin configuración para que los clientes localicen automáticamente su servidor de transmisión en la red.

Notas sobre estado y honestidad

- **Estado: beta.** Los objetivos de latencia del README (≤ 30 ms en LAN / ≤ 50 ms en WAN p999), el enfoque de “calidad consola / calidad PS4 Pro” y el número de celdas de la matriz de pruebas son metas de diseño declaradas por el propio

proyecto, no han sido verificadas de forma independiente y se presentan como tales.

- **Licencia: por determinar.** No se detectó ninguna LICENCIA mediante los códigos GitHub / API — SIN VERIFICAR / no declarada.

Nivel de prioridad: Helix-principal.